Sistema de Predicción de Riesgo Financiero — Documento del proyecto

Definición del proyecto

El proyecto consiste en un sistema informático de predicción de riesgo financiero, basado en analítica de datos y aprendizaje automático, que hoy está implementado de extremo a extremo en R. Procesa series de mercado (precios, volúmenes) y, de forma opcional, variables macroeconómicas para anticipar escenarios de alta volatilidad y pérdidas potenciales en carteras de inversión. Al integrar modelos estadísticos con un control de calidad y un backtesting explícitos, el sistema permite a operadores e inversionistas tomar decisiones informadas y aplicar medidas de mitigación antes de que se materialicen eventos adversos.

Esta versión del documento conserva la estructura original y la actualiza con lo ya construido y verificado: el ETL, la limpieza, el control de calidad, la ingeniería de características, el modelado predictivo y el backtesting, con resultados reales sobre SPY, AAPL y MSFT (2005–2026). El tablero interactivo y las alertas automáticas siguen pendientes y se indican como tales.

Objetivos y estado de cumplimiento

Objetivo general. Desarrollar e implementar un modelo predictivo automatizado capaz de estimar el riesgo de mercado y la probabilidad de pérdidas financieras, utilizando algoritmos de aprendizaje automático sobre datos históricos y flujos de información actualizados. Hoy se cumple para la volatilidad alta, con evidencia estadística; no se cumple para las caídas abruptas (ver Resultados).

La Tabla A resume cada objetivo específico y su estado real tras las pruebas con datos de Yahoo Finance.

Tabla A. Cumplimiento de los objetivos específicos

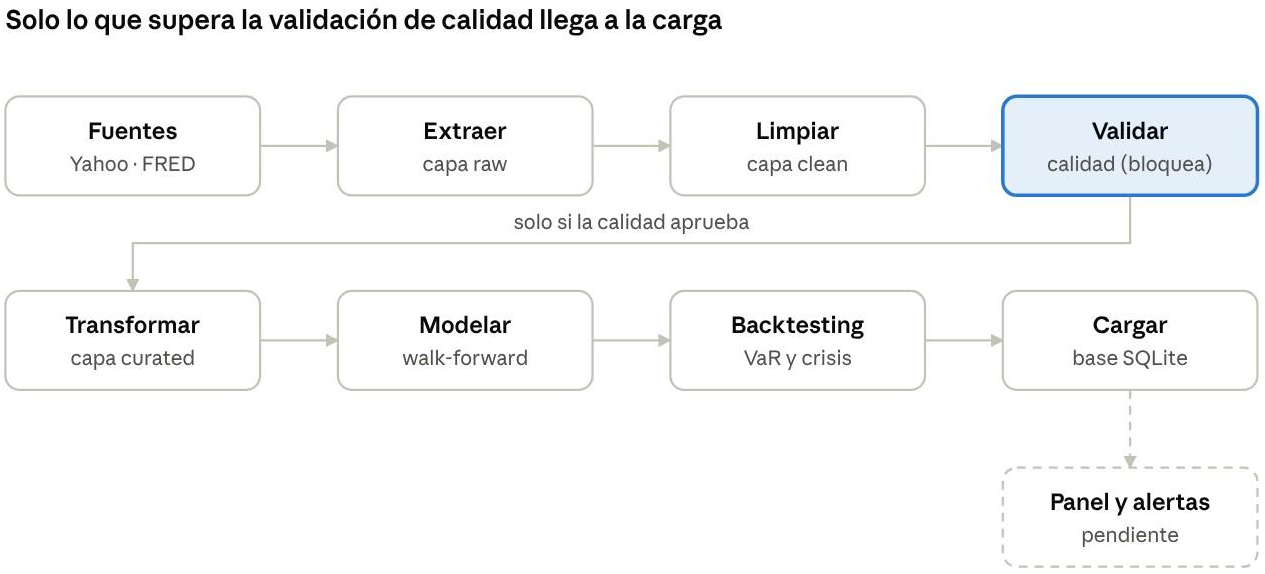
Objetivo específico	Estado	Evidencia
Recopilación y almacenamiento	Cumplido	Pipeline en R con capas raw/clean/curated y base SQLite; 5 493 filas por activo (2005-01-03 a 2026-11-02); carga idempotente.
Ingeniería de características	Cumplido	Retornos, medias móviles, volatilidad, VaR y ES (histórico y paramétrico), z-scores. VIX y demás variables macro requieren clave FRED (no ejecutado aún).

Modelado predictivo	Parcial	Regresión logística y bosque aleatorio con validación walk-forward. Volatilidad alta: señal útil y recomendada. Caídas abruptas: sin habilidad probabilística demostrada. Redes recurrentes: no implementadas.
Monitoreo y alertas	Pendiente	Umbrales de alerta ya calculados por modelo; falta el panel interactivo (Shiny) y el envío automático.
Evaluación de desempeño	Cumplido	Backtesting de VaR (Kupiec, Christoffersen, semáforo de Basilea) y de modelos en 2008 y 2020, con intervalos de confianza bootstrap.

Las decisiones de diseño que sostienen ese estado se explican en las secciones siguientes; los números completos están en Resultados reales y conclusiones.

Arquitectura del sistema

El flujo del diagrama original (fuentes, pipeline, base de datos, características, modelo, predicción, backtesting, panel y alertas) se conserva, pero ahora cada paso es una etapa ejecutable del pipeline en R.



pipeline en R · 7 etapas, 1 compuerta de calidad

Las etapas se ejecutan en este orden y cada una puede correrse por separado desde `pipeline.R` :

- **Extraer:** descarga las series y guarda la respuesta original sin tocarla (capa raw). **Limpiar:** aplica las
- reglas de la Tabla 2 y deja series estándar y sin registros inválidos (capa clean).

- **Validar:** evalúa las métricas de la Tabla 3; si una métrica bloqueante falla, el pipeline se detiene antes de cargar.
- **Transformar:** calcula retornos, medias móviles, volatilidad, VaR y Expected Shortfall (capa curated).
- **Modelar:** entrena y evalúa los modelos con validación walk-forward y genera las probabilidades de riesgo.
- **Backtesting:** contrasta el VaR y los modelos contra las crisis de 2008 y 2020. **Cargar:** escribe de
- forma idempotente en la base SQLite.

El panel interactivo y las alertas, que consumen la base, no están construidos todavía; por eso aparecen punteados.

Origen de los datos y APIs

Los datos provienen de fuentes complementarias para dar cobertura y robustez al modelo; de ellas, hoy están conectadas las de mercado y las macroeconómicas, y las alternativas quedan como extensión.

- **Datos de mercado (APIs financieras):** precios históricos y volúmenes. Se usa Yahoo Finance mediante el paquete `quantmod` (con un motor alterno `chart` que consulta la API pública directamente). Alpha Vantage queda configurado como fuente opcional (requiere clave y plan premium para precios ajustados); Bloomberg o la bolsa local no se integran.
- **Variables macroeconómicas:** tasas, inflación y tipo de cambio publicados por organismos oficiales, vía la API de FRED (VIX, bono a 10 años, tipo de cambio, tasa de fondos federales, IPC). Requieren una clave `FRED_API_KEY` ; las ejecuciones realizadas hasta ahora usaron la opción `--sin-macro` .
- **Datos alternativos:** noticias, sentimiento y eventos corporativos. No implementados; el diseño modular permite agregarlos como una fuente más.
- **Datos históricos de crisis:** las series de 2008 y 2020 se conservan íntegras (la serie descargada comienza en 2005) y se usan para entrenamiento y backtesting.

Activos y periodo reales. SPY, AAPL y MSFT, con 5 493 días de negociación por activo entre 2005-01-03 y 2026-11-02, calendario de la bolsa de Nueva York.

Prácticas del cliente de extracción. Cada cliente reintenta con espera creciente ante errores de red o límites de uso, oculta las claves en los mensajes de error y registra la marca de tiempo de descarga. Las credenciales se leen del archivo `.env` o de variables de entorno y nunca se guardan en el repositorio.

ETL del proceso y arquitectura por capas

El ETL (Extracción, Transformación y Carga) es la base operativa del pipeline y está implementado en R, con las mismas tres fases del diseño original.

1. **Extracción.** Conexión a las APIs mediante solicitudes programadas, con reintentos, que capturan datos históricos y recientes. La respuesta se guarda sin alteraciones.
2. **Transformación.**

- Limpieza: eliminación de duplicados, registros inválidos y días no hábiles; marcado de atípicos; tratamiento de nulos.
- Normalización y estandarización de las series (nombres, tipos, fechas en el día de la bolsa local).
- Cálculo de variables derivadas: retornos logarítmicos, medias móviles, volatilidad, VaR y Expected Shortfall.
- Alineación temporal de las fuentes, uniendo las variables macro por su fecha de publicación.

3. **Carga.** Almacenamiento estructurado en una base de datos relacional SQLite (en lugar de PostgreSQL o InfluxDB, que quedan como alternativa de escalamiento), disponible para el modelado y el panel. La carga es idempotente: reejecutarla para el mismo activo reemplaza sus filas dentro de una transacción y un índice único (ticker, fecha) impide duplicados.

Los datos originales nunca se modifican: toda corrección se hace sobre copias, lo que permite repetir el proceso con nuevas reglas sin volver a consultar las APIs.

Tabla 1. Capas de datos del ETL

Capa	Contenido	Almacenamiento	Uso
Raw (cruda)	Respuesta original de cada API, sin alteraciones, con marca de tiempo de descarga y fuente.	Archivos CSV por activo y fecha de extracción (`data/raw`).	Auditoría y reprocesamiento.
Clean (limpia)	Series estandarizadas, sin duplicados ni registros inválidos, con indicadores de calidad.	CSV particionado por activo y año (`data/clean/ticker=T/year=Y.csv`).	Entrada de la ingeniería de características.

Curated (curada)	Series alineadas, con variables derivadas (retornos, medias móviles, VaR, Expected Shortfall) y, luego, modelos y backtesting.	CSV (`data/curated`) y base SQLite (`data/riesgo.db`).	Entrenamiento, backtesting y panel.

La base SQLite contiene las tablas `precios_limpios` , `caracteristicas` , `predicciones` , `metricas_modelos` , `backtest_var` , `backtest_crisis` , `backtest_modelos_ic` y `reportes_calidad` .

Generación del ETL: limpieza de datos

La limpieza es el primer bloque de la fase de transformación y se diseña como un proceso formal, medible, reproducible y auditable, porque la calidad del modelo depende directamente de la de sus datos: duplicados, precios imposibles, huecos de calendario o macros mal alineadas sesgan el VaR y el Expected Shortfall y generan alertas falsas u omiten pérdidas reales.

Objetivos de la etapa

- **Garantizar integridad:** cada observación (fecha, activo) es única, está completa y tiene el tipo correcto.
- **Garantizar validez financiera:** se descartan registros que violan reglas básicas del mercado (precios menores o iguales a cero, máximo inferior al mínimo, cierre fuera del rango diario, volumen negativo).
- **Preservar la información de riesgo:** se distinguen los errores de captura de los eventos extremos reales, para no eliminar las caídas de 2008 y 2020 como si fueran ruido.
- **Evitar fuga de información (look-ahead bias):** los faltantes se tratan y las macros se alinean usando solo información disponible en cada fecha.
- **Asegurar trazabilidad:** cada ejecución genera un reporte cuantitativo con los registros corregidos, descartados y marcados.

Perfilado inicial

Antes de aplicar reglas se perfila cada fuente para tener una línea base: completitud por columna y activo, unicidad (fechas repetidas), rango y distribución (mínimos, máximos, percentiles), cobertura temporal (días hábiles esperados frente a recibidos) y frecuencia. En el código, la función `perfilar()` produce estas métricas y `evaluar_calidad()` las contrasta con los umbrales.

Reglas de limpieza

La Tabla 2 resume cada problema, su detección, el tratamiento aplicado y su justificación. Se marcan en negrita las reglas que cambiaron respecto al diseño original tras probarlas con datos reales.

Tabla 2. Reglas de limpieza aplicadas a las series financieras

Problema	Detección	Tratamiento	Justificación
Registros duplicados	Índice (fecha) repetido por activo.	Conservar el último registro recibido (orden estable, para que «el último» sea realmente el recibido).	La última versión suele incorporar correcciones del proveedor.
Formato y tipos inconsistentes	Fechas no interpretables, texto en columnas numéricas.	Conversión forzada; lo no convertible pasa a nulo.	Evita errores en cálculos posteriores.
Husos horarios distintos	Fechas con y sin zona horaria entre proveedores.	**Convertir a la fecha local del mercado (Nueva York), no a UTC.**	Evita desfases de un día al unir fuentes.
Precios imposibles	Precio ≤ 0; máximo < mínimo; cierre fuera de [mínimo, máximo]; volumen < 0.	Invalidar el registro (nulo) y registrarlo en el reporte.	Son errores de captura, no movimientos de mercado.
Valores nulos	Celdas vacías o producidas por la invalidación.	Propagar el último precio válido (máximo 3 días); si persiste, descartar la fila. **El calendario se completa solo en días hábiles reales.**	Cubre fallos breves sin inventar tendencias; no hay relleno hacia atrás.
Valores atípicos	**Z-score robusto (mediana y MAD) sobre retornos logarítmicos de `adj_close`, con ventana móvil de 60 días únicamente pasada y umbral de 8.**	Marcar siempre; tratar como error de dato solo si **el salto se revierte al día siguiente, es ≥ 0,405 en log-retorno y no viene con volumen ≥ 3 veces la mediana**.	Una caída persistente es una crisis real; un salto que se revierte es un error.

Días no hábiles	Fines de semana o feriados bursátiles.	Excluir del calendario NYSE (con feriados extra configurables).	Evita retornos artificiales de valor cero.
Splits y dividendos	Saltos de precio sin correspondencia con el volumen.	Usar precio ajustado (`adj_close`) para los retornos.	Un split no es una pérdida financiera.
Problema	Detección	Tratamiento	Justificación
Variables macro con rezago	Frecuencia mensual o trimestral y fecha de publicación posterior al periodo.	Alinear por fecha de publicación (rezago por serie o primera publicación/vintage) con unión hacia atrás.	Evita look-ahead bias.

Tratamiento de valores atípicos

El tratamiento de atípicos es la decisión metodológica más delicada: las colas pesadas son una característica genuina de los mercados, y un filtro clásico (por ejemplo, descartar todo lo que supere tres desviaciones estándar) eliminaría justamente los eventos que el sistema debe anticipar. Se adopta por eso un criterio en dos pasos. Primero, un z-score robusto basado en la mediana y la MAD de una ventana solo pasada, menos sensible a los propios extremos. Segundo, un extremo se clasifica como error de dato solo si el retorno del día siguiente lo compensa casi por completo. Los extremos que persisten se conservan y se marcan con `es_atipico` .

La prueba con datos reales de SPY mostro que esa regla, tal como estaba, tenía un falso positivo: un rebote legítimo del 12 de marzo de 2020 se trataba como error. Por eso se agregaron los dos filtros adicionales (tamaño mínimo del salto y guía de volumen). Con ellos, el resultado sobre las series reales fue:

Tabla B. Atípicos detectados en los datos reales (2005–2026)

Activo	Atípicos marcados	Errores de dato
SPY	9 (entre ellos 2007-02-27, 2018-02-05, 2018-02-08, 2020-03-09/12/13/16/24 y 2025-04-09)	0
AAPL	2	0
MSFT	7	0

Todos los atípicos de SPY coinciden con eventos reales (la caída de febrero de 2018, el pánico de marzo de 2020, los aranceles de abril de 2025), por lo que se conservan.

Implementación del módulo de limpieza

El Código 1 presenta, en R, las dos funciones centrales (`R/limpieza.R`). Cada regla es una función independiente, lo que facilita las pruebas unitarias y la reutilización con otras fuentes.

Código 1. R/limpieza.R (fragmento)

```
marcar_atipicos <- function(df, ventana = 60L, umbral = 8, tol = 0.25,
min_periodos = 20L,
                            salto_minimo_error = 0.405, factor_volumen = 3) {
precio <- if ("adj_close" %in% names(df)) df$adj_close else df$close
  precio[!is.na(precio) & precio <= 0] <- NA
r <- c(NA_real_, diff(log(precio)))

  mm <- mediana_mad_previa(r, ventana, min_periodos)   # solo datos pasados
  mad <- ifelse(!is.na(mm$mad) & mm$mad > 0, mm$mad, NA_real_)
  z <- 0.6745 * (r - mm$mediana) / mad
extremo <- !is.na(z) & abs(z) > umbral

  r_sig <- c(r[-1], NA_real_)
  reversion <- !is.na(r + r_sig) & abs(r + r_sig) < tol * abs(r)
desproporcionado <- !is.na(r) & abs(r) >= salto_minimo_error

  volumen_real <- rep(FALSE, nrow(df))
if ("volume" %in% names(df)) {
    previo <- mediana_mad_previa(df$volume, ventana, min_periodos)$mediana
volumen_real <- !is.na(df$volume) & !is.na(previo) & df$volume >=
factor_volumen * previo
  }
  df$es_atipico <- extremo
  df$error_dato <- extremo & reversion & desproporcionado & !volumen_real
df }

limpiar_serie <- function(df, ticker, params, zona_horaria = NULL, calendario
= NULL, fuente = NA) {
  e <- estandarizar_esquema(df, zona_horaria); df <- e$df   r <-
eliminar_duplicados(df);                      df <- r$df   r <-
excluir_no_habiles(df, calendario);           df <- r$df   r <-
validar_coherencia_ohlc(df, params$tolerancia_ohlc); df <- r$df   r
<- completar_calendario(df, calendario);         df <- r$df   df <-
marcar_atipicos(df, params$ventana, params$umbral_atipico,
params$tolerancia_reversion,
                        params$min_periodos, params$salto_minimo_error,
params$factor_volumen_real)
  for (col in intersect(COLUMNAS_PRECIO, names(df))) df[[col]][df$error_dato]
<- NA_real_
  r <- tratar_nulos(df, params$limite_ffill);        df <- r$df
  list(df = df, reporte = reporte_de_la_corrida)   # contadores por regla
}
```

La alineación de variables macro (`alinear_macro()`) usa `findInterval()` sobre la fecha en que cada dato estuvo disponible, de modo que cada día solo ve el último valor ya publicado.

Orquestación y automatización

El proyecto ya no usa Apache Airflow ni Python: las tareas se encadenan con un orquestador propio en R (`R/tareas.R`) y la ejecución periódica se delega al programador del sistema operativo. El diseño conserva lo esencial del esquema original: etapas ordenadas, reintentos ante fallos temporales y detención antes de la carga si la calidad no aprueba.

Código 2. Ejecución del pipeline (pipeline.R)

```
# Pipeline completo con la configuración de config/config.yaml
Rscript pipeline.R

# Solo precios de Yahoo (no requiere clave FRED)
Rscript pipeline.R --sin-macro

# Datos sintéticos, sin APIs ni claves
Rscript pipeline.R --demo

# Reprocesar sin volver a descargar
Rscript pipeline.R --etapas limpiar validar transformar cargar

# Programar la ejecución diaria en Windows (Programador de tareas)
programar_windows.bat
```

Horario de ejecución. El documento original proponía las 22:00 tras el cierre, pero eso contradice el umbral de oportunidad de la Tabla 3 (≤ 2 horas desde el cierre). El horario queda aproximadamente 1 h 30 min después del cierre (16:00 en Nueva York), de lunes a viernes: la configuración lo expresa como `30 17 * * 1-5` y `programar_windows.bat` lo registra con la hora local del equipo (16:30 en Ciudad de México durante el invierno de EE. UU.).

Además del orquestador, el proceso incorpora estas prácticas de robustez:

- **Idempotencia:** ejecutar dos veces el proceso para la misma fecha produce el mismo resultado y no duplica registros; la carga por activo ocurre dentro de una transacción y con un índice único.
- **Reintentos con espera:** ante errores de red o límites de uso de las APIs se reintenta hasta tres veces con intervalos crecientes.
- **Registro de eventos (logging):** cada función registra las filas procesadas, corregidas y descartadas, y el reporte queda guardado en `reportes_calidad` .
- **Configuración externa:** ventana, umbral, límite de relleno, rezagos, parámetros de modelado y backtesting se leen de `config/config.yaml` , no del código.

- **Gestión de credenciales:** las claves (`FRED_API_KEY` , `ALPHAVANTAGE_API_KEY`) viven en `.env` o en variables de entorno, nunca en el repositorio.
- **Ruta de datos separada del código:** la variable `ETL_DATOS` apunta a una carpeta de datos fuera del proyecto, de modo que actualizar el código no borra los datos descargados.

Control de calidad y validaciones

Al finalizar la limpieza se ejecutan validaciones automáticas (`R/calidad.R`): si una métrica bloqueante incumple su umbral, el flujo se detiene antes de la carga y avisa, de modo que ningún dato defectuoso llegue al modelo. Las métricas no bloqueantes generan alertas en el reporte.

Tabla 3. Dimensiones de calidad, métricas y umbrales de aceptación

Dimensión	Métrica	Umbral de aceptación	Efecto
Completitud	Porcentaje de días hábiles con precio de cierre válido por activo.	≥ 99 %	Bloquea
Unicidad	Fechas duplicadas por activo tras la limpieza.	0	Bloquea
Validez	Registros que incumplen las reglas de coherencia OHLC.	0	Bloquea
Consistencia	Series con fechas fuera de orden o con huecos mayores a 3 días hábiles.	0	Bloquea
Oportunidad	Retraso entre el cierre del mercado y la disponibilidad del dato limpio.	≤ 2 horas	Desactivada por defecto (los datos históricos no tienen un cierre reciente); se activa en la ejecución diaria.
Dimensión	Métrica	Umbral de aceptación	Efecto

Impacto de la limpieza	Porcentaje de filas modificadas o descartadas respecto de la entrada.	Alerta si > 2 %	Solo alerta

Nota: los umbrales son valores iniciales de referencia y pueden ajustarse según las características de cada fuente tras el perfilado. Viven en `config/config.yaml`.

El impacto se mide con las filas realmente eliminadas, corregidas o rellenadas (duplicados, días no hábiles, filas sin precio y rellenos), por lo que no cuenta como «modificación» el simple marcado de un atípico. Con los datos reales de Yahoo, las tres series pasaron todas las validaciones bloqueantes y el pipeline llegó a la carga.

Validación del módulo con datos sintéticos y pruebas

Para verificar el código se usa una serie sintética de 1 000 días hábiles a la que se inyectan anomalías conocidas: dos fechas duplicadas, dos precios negativos o incoherentes, dos valores nulos consecutivos, un error de captura aislado (un precio multiplicado por diez durante un solo día) y una caída persistente del 30 % que simula una crisis real. La serie se genera con `R/sintetico.R` y también permite ejecutar todo el pipeline sin APIs (`-demo`).

Tabla 4. Resultado de la prueba sobre la serie sintética

Anomalía inyectada	Resultado del módulo
2 fechas duplicadas	Detectadas y eliminadas (duplicados = 2).
2 registros incoherentes (precio negativo y máximo menor que el mínimo)	Detectados e invalidados (incoherentes = 2).
2 valores nulos consecutivos	Cubiertos con el último precio válido; no se perdió ninguna fila.
1 error de captura (precio × 10 durante un día)	Clasificado como error de dato y reemplazado por el último precio válido (errores_dato = 1).
Caída persistente del 30 %	Marcada como atípica pero conservada; el nivel posterior de la serie no se alteró.
Anomalía inyectada	Resultado del módulo

Filas de entrada / salida	1002 / 1000 (se eliminaron únicamente los duplicados).

Este resultado confirma el comportamiento esperado: el módulo corrige los errores de captura sin eliminar los eventos extremos legítimos y deja constancia cuantitativa de cada intervención en el reporte de calidad.

Pruebas automatizadas

La verificación no se limita a la serie sintética: el proyecto tiene 89 pruebas unitarias y de integración con `testthat` , repartidas en seis archivos (limpieza, extracción, pipeline, visualización, modelado y backtesting), y todas pasan en R 4.3.3 (entorno de desarrollo); en R 4.6.2 sobre Windows se verificó la versión anterior de 62 pruebas. Se ejecutan con:

```
Rscript tests/testthat.R
```

Además de las funciones de limpieza, las pruebas cubren el análisis de modelos (sin fuga de información y con caso de respaldo cuando hay pocos eventos) y las pruebas estadísticas del backtesting.

Ingeniería de características

La capa curada se construye con `R/transformacion.R` a partir de la serie limpia y contiene solo variables calculadas con información disponible hasta cada fecha. Las medidas de riesgo se definen así.

Retorno logarítmico del precio ajustado:

```
r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)
```

Volatilidad histórica a 21 días, anualizada:

```
\sigma_t = \sqrt{252}\;\mathrm{sd}\left(r_{t-20},\dots,r_t\right)
```

Valor en Riesgo (VaR) histórico al nivel α, con una ventana móvil de retornos pasados (se reporta como pérdida positiva):

```
\mathrm{VaR}_{\alpha,t} = -\,Q_{1-\alpha}\left(r_{t-W},\dots,r_{t-1}\right)
```

Expected Shortfall (ES), la pérdida media cuando se supera el VaR:

```
\mathrm{ES}_{\alpha,t} = -\,\mathbb{E}\left[\,r_s \mid r_s \le -
\mathrm{VaR}_{\alpha,t}\right]
```

VaR paramétrico (normal), con media μ y desviación σ de la misma ventana y zα el cuantil normal:

```
\mathrm{VaR}^{\mathrm{param}}_{\alpha,t} = -\left(\mu_t - z_{\alpha}\,\sigma_t\right)
```

Los niveles de confianza calculados son 95 % y 99 %.

Variables resultantes en la capa curada

Grupo	Variables
Retornos y tendencia	retorno logarítmico, medias móviles de 20, 50 y 200 días
Volatilidad	volatilidad histórica de 21 días anualizada, z-scores móviles
Riesgo	VaR histórico y paramétrico (95 % y 99 %), Expected Shortfall
Macroeconómicas	VIX, bono a 10 años, tipo de cambio, tasa de fondos federales e IPC alineados por fecha de publicación (solo si se configura la clave FRED)

La volatilidad implícita del documento original se aproxima con el VIX, que depende de la clave FRED; mientras no se ejecute con macro, el modelo usa solo la volatilidad histórica.

Riesgo más reciente medido (2026-11-02).

Activo	VaR 95 %	ES 95 %	VaR 99 %	Volatilidad anualizada
SPY	1,43 %	1,72 %	1,80 %	11,4 %
AAPL	2,03 %	3,55 %	4,43 %	15,8 %
MSFT	2,83 %	4,15 %	4,16 %	22,2 %

Modelado predictivo

Los modelos funcionan para anticipar volatilidad alta, pero no para predecir caídas abruptas: esa es la conclusión central de esta etapa y se apoya en validación fuera de muestra y comparación contra una regla simple.

Qué se predice

- **Volatilidad alta (`y_vol_alta`):** la volatilidad realizada de los próximos 5 días supera el percentil 90 de su propio pasado (calculado solo con datos anteriores, recalculado cada 10 días y con 250 días de historia mínima).
- **Caída abrupta (`y_caida`):** el retorno de los próximos 5 días es menor o igual a −5 %.

Cómo se entrena y se evita la fuga de información

- Todas las variables usan solo información pasada; se añadieron variables de volatilidad relativa (21 y 5 días frente al umbral de la etiqueta).
- Validación **walk-forward**: ventana inicial de 1 260 días, avance de 252 días y purga de 10 días (mayor al horizonte de 5) entre entrenamiento y prueba.
- **Regresión logística** con red elástica (alfa en {0; 0,5; 1}, elegida por devianza, lambda mínimo).
- **Bosque aleatorio** (`ra` con búsqueda de mtry {3, 6} y tamaño mínimo de nodo {25, 100} por Brier, y calibración de Platt.
- Si el entrenamiento tiene menos de 10 eventos, se usa un modelo constante de respaldo.
- Las **redes neuronales recurrentes** del objetivo original no se implementaron: con tres activos y pocos eventos, un modelo más pesado no tendría datos suficientes para justificarse.

Cómo se mide

Se reportan AUC y Brier, la **habilidad de Brier** frente a la frecuencia histórica del evento (positiva = mejor que no saber nada) y una **regla base** (volatilidad histórica de 21 días para la volatilidad; VaR histórico 95 % para caídas). Un modelo se marca como `recomendado` solo si su habilidad de Brier es positiva. El umbral de alerta maximiza F1 con una tasa de alertas no mayor al 25 %.

Resultados fuera de muestra

Se evaluaron 3 974 días para volatilidad y 4 228 para caídas.

Tabla C. Volatilidad alta, resultados fuera de muestra

Activo	Modelo	AUC	AUC regla base	Habilidad Brier	Precisión	Recall	Recomendado
SPY	Logística	0,862	0,834	+27,5 %	0,403	0,544	Sí
SPY	Bosque	0,784	0,834	+13,6 %			
AAPL	Logística	0,808	0,758	+14,7 %	0,372	0,225	Sí
AAPL	Bosque	0,671	0,758	+3,8 %			
MSFT	Logística	0,731	0,706	+11,2 %	0,287	0,374	Sí
MSFT	Bosque	0,710	0,706	+7,5 %			

Tabla D. Caídas abruptas (−5 % en 5 días), resultados fuera de muestra

Activo	Modelo	AUC	AUC regla base	Habilidad Brier
SPY	Logística	0,724	0,557	+0,8 % (no significativa)
SPY	Bosque	0,560	0,557	−0,6 %

AAPL	Logística	0,594	0,528	−0,2 %
AAPL	Bosque	0,536	0,528	−0,1 %
MSFT	Logística	0,652	0,553	−1,3 %
MSFT	Bosque	0,548	0,553	−0,3 %

Lectura de los resultados

- **Volatilidad alta:** la regresión logística supera a la regla base en los tres activos y tiene habilidad de Brier positiva; por eso queda como recomendada. En SPY, con un AUC de 0,862, el modelo detecta algo más de la mitad de los episodios con alrededor de 4 aciertos por cada 10 alertas.
- **El bosque aleatorio no aporta:** rinde por debajo de la logística en los tres activos y, en AAPL, incluso peor que la regla base. Se mantiene como comparación, no como modelo recomendado.
- **Caídas abruptas:** el AUC supera a la regla base, es decir, los modelos ordenan mejor los días, pero las probabilidades no están bien calibradas (habilidad de Brier ≤ 1 %). Con tan pocos eventos, no se recomienda usarlos para alertas de caídas.
- **Señales actuales:** el 2026-11-02 ningún modelo recomendado emite alerta; por ejemplo, la probabilidad de volatilidad alta en SPY es 2,7 % frente a un umbral de alerta de 21,8 %.

Backtesting y escenarios de crisis

El backtesting mide dos cosas distintas: si el VaR calculado cumple su nivel de confianza y si los modelos habrían anticipado las crisis de 2008 y 2020. El primero casi no pasa; el segundo funciona para volatilidad alta y no para caídas.

Backtesting del VaR

Se cuentan las violaciones (días en que la pérdida superó el VaR) y se aplican:

- **Prueba de Kupiec (POF):** si la tasa de violaciones coincide con la esperada (5 % o 1 %). **Prueba de**
- **independencia de Christoffersen:** si las violaciones se agrupan en el tiempo.
- **Cobertura condicional:** la combinación de ambas.
- **Semáforo de Basilea** (99 %): verde 0 a 4 violaciones por 250 días, amarillo 5 a 9, rojo 10 o más.
- **Razón del ES:** pérdida media en las violaciones frente al ES estimado.

Tabla E. VaR histórico: violaciones y resultado de las pruebas

Activo	Nivel	Periodo	Violaciones	Tasa	Resultado
SPY	95 %	Todo el periodo	285	5,4 %	Rechazado (independencia p < 0,001; Kupiec p = 0,152)

SPY	95 %	2008		10,5 %	Rechazado
SPY	95 %	COVID		17,2 %	Rechazado
SPY	99 %	Todo el periodo	89	1,7 %	Rechazado; semáforo rojo
SPY	99 %	2008	11	5,3 %	Rechazado
SPY	99 %	COVID	8	8,6 %	Rechazado
AAPL	99 %	Todo el periodo	81	1,5 %	Semáforo amarillo
AAPL	99 %	2008			No rechazado (p = 0,238 y 0,692)
AAPL	99 %	COVID		5,4 %	Rechazado
MSFT	99 %	Todo el periodo	90	1,7 %	Semáforo rojo
MSFT	99 %	2008		3,3 %	
MSFT	99 %	COVID		7,5 %	

En total, 16 de las 18 combinaciones (activo, nivel, periodo) se rechazan; solo AAPL en 2008 supera las pruebas. El VaR histórico de ventana móvil reacciona tarde: subestima el riesgo cuando la volatilidad sube de golpe y por eso las violaciones se agrupan en las crisis. Esto es justamente lo que motiva el uso de modelos que anticipen la volatilidad.

Backtesting de los modelos en crisis

Para cada crisis el modelo se entrena solo con datos anteriores a ella (menos la purga) y se mide qué tan temprano habría alertado. La habilidad de Brier se compara con la frecuencia previa a la crisis, por lo que sale inflada: se interpreta junto con el AUC y las alertas, no sola.

Tabla F. Volatilidad alta en crisis: modelo logístico

Activo	Crisis	AUC	Habilidad Brier	Alertas	Precisión	Recall	Primera alerta	Anticipación
SPY	2008	0,865	+55 %	129	0,81	0,85	2008-09-04	29 días
SPY	COVID	0,802	+63 %		0,67	0,80	2020-02-24	10 días
AAPL	2008	0,951	+29 %		0,95		2008-09-16	10 días
AAPL	COVID	0,873	+63 %		0,85		2020-02-25	17 días
MSFT	2008	0,735	+36 %	201 de 209	0,53			

MSFT	COVID	0,852	+51 %		0,86		2020-02-24	10 días

En SPY, la regla base (volatilidad histórica) alcanzó un AUC de 0,774 en 2008 con 209 alertas y precisión de 0,59, y de 0,720 en COVID con su primera alerta el 2 de marzo, es decir, más tarde que el modelo (24 de febrero). El bosque aleatorio tuvo AUC similar en SPY (0,849 y 0,803) pero casi no alertó en 2008 (2 alertas). En MSFT 2008 el modelo alerta en 201 de 209 días, lo que lo vuelve poco informativo.

Caídas abruptas en crisis. En 2008 los modelos quedan constantes (AUC 0,5 y ninguna alerta) porque el entrenamiento no contiene caídas previas; en COVID el AUC fluctúa entre 0,51 y 0,76, la habilidad de Brier es cercana a cero o negativa (hasta −45 % en MSFT con logística) y la precisión de 0,12 a 0,26. Confirma que las caídas bruscas no son predecibles con estas variables.

Intervalos de confianza (bootstrap por bloques)

Para no sobreinterpretar una muestra corta se calcularon intervalos con bootstrap por bloques móviles de 21 días y 500 remuestreos.

Tabla G. Volatilidad alta: intervalos de confianza al 95 %

Activo	Modelo	AUC [IC]	Diferencia de AUC frente a regla base [IC]	Habilidad Brier [IC]
SPY	Logística	0,862 [0,802; 0,916]	+0,029 [−0,003; 0,061]	+27,5 % [12,6; 42,6]
AAPL	Logística		+0,050 [0,011; 0,095]	+14,7 % [2,0; 27,5]
AAPL	Bosque		−0,086 [−0,166; −0,023]	
MSFT	Logística		+0,024 [−0,002; 0,054]	+11,2 % [2,2; 21,0]

La habilidad de Brier de la logística es significativamente positiva en los tres activos (el intervalo excluye cero), y su ventaja de AUC frente a la regla base es significativa solo en AAPL; en SPY y MSFT es favorable pero el intervalo roza cero. Para las caídas, la diferencia de AUC frente a la regla base es significativa (SPY +0,167 [0,078; 0,236], AAPL +0,066 [0,006; 0,141], MSFT +0,099 [0,021; 0,176]), pero la habilidad de Brier no lo es en ningún activo.

Resultados reales y conclusiones

Con datos reales de 2005 a 2026, el sistema anticipa bien la volatilidad alta y no anticipa las caídas abruptas; las demás conclusiones se derivan de esa diferencia.

- **Datos y calidad:** 5 493 días por activo, 18 atípicos marcados en total (9 en SPY, 2 en AAPL, 7 en MSFT) y ningún error de dato; las validaciones bloqueantes se superaron. • **Volatilidad alta:** la regresión logística obtiene AUC de 0,73 a 0,86 y habilidad de Brier positiva y estadísticamente significativa en los tres activos. En las crisis alertó con 10 a 29 días de anticipación en 5 de los 6 casos con alertas informativas (SPY y AAPL en ambas crisis y MSFT en COVID).
- **Caídas abruptas:** hay poca habilidad probabilística; no deben usarse para alertar.
- **VaR histórico:** no cumple su cobertura en la mayoría de los casos y subestima el riesgo en las crisis; sirve como referencia, no como límite confiable.
- **Bosque aleatorio:** no mejora a la logística; la mayor complejidad no se justifica con estos datos.

Veredicto por objetivo. Se cumplen el pipeline de datos, las características y la evaluación con backtesting. El modelado se cumple en volatilidad y no en caídas. El monitoreo con alertas automáticas está pendiente.

Hallazgos y correcciones durante las pruebas

Probar el sistema con datos reales y no solo sintéticos destapó errores que la versión original del diseño no contemplaba; los más relevantes se corrigieron así.

Hallazgo	Corrección
El orden de las filas no era estable, por lo que «conservar el último duplicado» podía conservar el primero.	Ordenación estable.
Fechas con zona horaria mezcladas convertían registros válidos en nulos sin aviso.	Conversión a la fecha local del mercado, con aviso si aparecen nulos nuevos.
La regla de atípicos marcó como error un rebote real del 12-mar-2020 en SPY.	Se añadieron el tamaño mínimo del salto y la guía de volumen.
El z-score usaba una ventana que incluía el propio día.	Ventana móvil solo pasada.
Los modelos iniciales no superaban a la regla base.	Criterios por devianza y Brier, calibración de Platt y variables de volatilidad relativa; se retiraron variables redundantes.
El umbral de alerta podía disparar todos los días.	Límite del 25 % de alertas y ninguna alerta para predicciones constantes.
Configuración en YAML con acentos leída como vacía en Windows.	Lectura explícita como UTF-8 y validación de la configuración.

Los datos se perdían al actualizar el código.	Variable `ETL_DATOS` con una carpeta de datos independiente.
El horario de 22:00 incumplía la métrica de oportunidad.	Horario de unos 90 minutos tras el cierre.

Limitaciones y trabajo pendiente

Los resultados son válidos para lo medido, pero hay límites claros y trabajo por terminar.

Limitaciones de la evidencia

- Solo tres activos de EE. UU. (SPY, AAPL, MSFT) y dos crisis; la ventana de COVID tiene apenas 93 días, por lo que los resultados de crisis son indicativos, no concluyentes.
- Las caídas abruptas son eventos raros: 2008 no tiene caídas previas para entrenar y los modelos quedan constantes.
- La habilidad de Brier en crisis se mide frente a la frecuencia previa a la crisis y está inflada; no debe leerse sola.
- Las variables macro (VIX, tasas) aún no se probaron; podrían mejorar o no el modelo. Redes
- neuronales recurrentes y datos alternativos (noticias, sentimiento) no están implementados.

Pendiente por construir

- **Panel interactivo (Shiny)** con los indicadores de riesgo de cada activo.
- **Alertas automáticas** por umbral (VaR, volatilidad y probabilidad del modelo), usando solo los modelos marcados como recomendados. Queda por decidir si también se enviarán por correo.
- **Ejecución con clave FRED** para incluir el VIX y reevaluar los modelos.
- Ampliar el universo de activos y reemplazar el código en Python del repositorio de GitHub por esta versión en R.

Documentación en Confluence

La documentación del proyecto se centraliza en Confluence, que funciona como fuente única de verdad para el equipo. En un sistema con varios componentes (ingesta, limpieza, modelos, panel y alertas), documentar de forma estructurada permite que otras personas entiendan, mantengan y reproduzcan el trabajo, y que el proyecto se evalúe con transparencia. Las páginas técnicas se actualizan con lo ya implementado en R.

Objetivos de la documentación

- **Centralizar el conocimiento:** reunir en un solo espacio las decisiones técnicas, los procedimientos y los resultados.

- **Facilitar la incorporación de nuevos integrantes:** que una persona nueva comprenda la arquitectura y ejecute el ETL siguiendo solo las páginas del espacio.
- **Garantizar la reproducibilidad:** describir versiones, parámetros y dependencias para repetir los resultados.
- **Dar trazabilidad a las decisiones:** registrar por qué se eligió una técnica y qué alternativas se descartaron (por ejemplo, el criterio de atípicos, o por qué se dejó Python).
- **Apoyar la evaluación del proyecto:** ofrecer evidencia consultable del avance y de la calidad del trabajo.

Estructura del espacio

Se crea un espacio denominado «Sistema de Predicción de Riesgo Financiero» con una jerarquía de páginas que sigue el flujo de datos. **Tabla 5. Árbol de páginas del espacio en Confluence**

Página	Contenido principal
Inicio del proyecto	Resumen ejecutivo, objetivos, equipo, enlaces rápidos, estado general y calendario de entregas.
1. Definición y alcance	Definición del proyecto, objetivo general y específicos, restricciones y supuestos (incluida la restricción de no usar Python).
2. Arquitectura	Diagrama general del sistema, componentes, tecnologías (R) y flujo de datos.
3. Fuentes de datos	Catálogo de APIs y fuentes (mercado, macro, alternativas, crisis), formatos, frecuencia, límites de uso y licencias.
4. ETL	Subpáginas: Extracción, Limpieza, Transformación y Carga. Incluye reglas, parámetros, ejemplos y manejo de errores.
5. Ingeniería de características	Definición matemática de VaR, Expected Shortfall, volatilidad, retornos y medias móviles.
6. Modelado predictivo	Algoritmos evaluados, hiperparámetros, métricas y resultados comparativos.
7. Panel y alertas	Indicadores, umbrales de tolerancia, reglas de alerta y guía de usuario (pendiente).

8. Backtesting	Escenarios de crisis, metodología de validación y resultados.
9. Guías operativas	Instalación, ejecución, monitoreo y solución de problemas.
10. Registro de decisiones	Decisiones técnicas con contexto, alternativas y consecuencias.
11. Reuniones y avances	Actas, acuerdos y acciones pendientes.
12. Glosario	Términos financieros y técnicos.

Plantilla de página técnica del ETL

Todas las páginas de la sección ETL siguen una plantilla común:

- **Propósito:** qué problema resuelve el componente y en qué punto del flujo se ubica.
- **Diagrama de flujo:** entradas, procesos y salidas.
- **Entradas y salidas:** fuentes, formatos, esquemas de datos y ubicación de los archivos generados.
- **Reglas de negocio y de calidad:** las reglas de limpieza (Tabla 2) y los umbrales de aceptación (Tabla 3).
- **Parámetros de configuración:** ventana móvil, umbral de atípicos, límite de relleno, rezagos y frecuencia de ejecución (`config/config.yaml`).
- **Dependencias:** versión de R y paquetes (yaml, DBI, RSQLite, jsonlite, timeDate, zoo, httr2, quantmod, glmnet, ranger, ggplot2, testthat).
- **Ejemplo de ejecución:** comando `Rscript pipeline.R` y un ejemplo de resultado.
- **Manejo de errores:** fallos conocidos, mensajes frecuentes y pasos de solución (por ejemplo, «No hay capa curated para SPY»: ejecutar primero el pipeline o revisar `ETL_DATOS`).
- **Responsable e historial:** persona a cargo, fecha de última revisión y registro de cambios.

Diccionario de datos

Se mantiene un diccionario de la capa limpia que describe cada campo y su regla de calidad.

Tabla 6. Diccionario de datos de la capa limpia (series de precios)

Campo	Tipo	Descripción	Regla de limpieza
fecha	Fecha	Día de negociación en la fecha local del mercado.	Única por activo; sin fines de semana ni feriados.
ticker	Texto	Identificador del activo.	No nulo.

open, high, low, close	Decimal	Precios de apertura, máximo, mínimo y cierre.	Mayores que cero; low ≤ close ≤ high.
adj_close	Decimal	Cierre ajustado por splits y dividendos.	Mayor que cero; base de los retornos.
volume	Entero	Volumen negociado.	Mayor o igual a cero.
Campo	Tipo	Descripción	Regla de limpieza
es_atipico	Lógico	Indica retorno extremo según z-score robusto.	Se conserva como atributo informativo.
error_dato	Lógico	Salto extremo revertido al día siguiente.	Valor corregido con el último precio válido.
relleno	Lógico	La fila se completó con el último precio válido.	Máximo 3 días consecutivos.
fuente	Texto	Proveedor de origen del dato.	No nulo; trazabilidad.

Uso de funcionalidades de Confluence

- **Tabla de contenidos:** macro que genera el índice de cada página extensa a partir de sus encabezados.
- **Bloques de código:** macro con resaltado de sintaxis para los scripts en R y las consultas SQL.
- **Paneles de información, nota y advertencia:** para destacar supuestos, riesgos y precauciones (por ejemplo, el límite de uso de una API).
- **Propiedades de página y su informe:** resumen automático de estado de las páginas del ETL.
- **Etiquetas (labels):** para clasificar contenido (etl, limpieza, modelo, api) y facilitar la búsqueda.
- **Integración con Jira:** para vincular cada tarea con la página que la documenta.
- **Historial de versiones:** para consultar, comparar y restaurar cambios.
- **Comentarios en línea:** para revisiones entre compañeros sin alterar el contenido original.

Vinculación con el repositorio Git

La documentación y el código se mantienen enlazados. Cada página técnica del ETL incluye el vínculo al módulo del repositorio, a la rama y a la versión (etiqueta) con la que se obtuvieron los resultados descritos, y el README del repositorio apunta al espacio de Confluence. Así, quien consulte el código encuentra su explicación y quien lea la documentación llega a la implementación exacta.

Gobierno de la documentación

Tabla 7. Roles y responsabilidades

Rol	Responsabilidad
Autor	Redactar y actualizar la página del componente que desarrolló.
Revisor	Verificar claridad, exactitud técnica y cumplimiento de la plantilla antes de marcar la página como aprobada.
Coordinador del proyecto	Mantener la estructura del espacio, gestionar permisos y vigilar que la documentación esté al día.

Una tarea no se considera terminada hasta que su documentación esté publicada y revisada (definición de terminado). Cada página lleva un estado visible (borrador, en revisión o aprobada) y se revisa al menos al cierre de cada iteración o cuando cambie el componente que describe.

Criterios de aceptación de la documentación

- **Completitud:** todos los componentes del sistema cuentan con su página, siguiendo la plantilla.
- **Claridad:** una persona ajena al desarrollo puede ejecutar el ETL siguiendo solo la guía operativa.
- **Actualidad:** los parámetros, diagramas y diccionarios corresponden a la versión vigente del código.
- **Trazabilidad:** cada decisión relevante está registrada con su justificación. **Accesibilidad:** los
- permisos de lectura y edición están configurados para todo el equipo y las personas evaluadoras.

Guía de ejecución y estructura del repositorio

Cualquier persona con R 4.x puede reproducir los resultados siguiendo estos pasos desde la carpeta del proyecto (probado en Windows con R 4.6.2).

1. **Instalar los paquetes.**

```
Rscript instalar_paquetes.R
```

2. **Indicar dónde se guardan los datos** (fuera del código, para que actualizarlo no los borre) y, si se desea, las claves en el archivo `.env` :

```
echo ETL_DATOS=C:\Users\<usuario>\datos_riesgo> .env
```

3. **Ejecutar el pipeline completo** (descarga, limpieza, calidad, características, modelos, backtesting y carga):

```
Rscript pipeline.R --sin-macro
```

4. **Revisar los resultados:**

```
Rscript ver_datos.R       # datos limpios y riesgo más reciente
Rscript ver_modelos.R     # desempeño fuera de muestra y señales de hoy
Rscript ver_backtest.R    # backtesting del VaR y de crisis, con intervalos
Rscript visualizar.R      # gráficas (panel de riesgo por activo,
comparación, backtesting)
```

5. **Ejecutar las pruebas y programar la ejecución diaria:**

```
Rscript tests/testthat.R
programar_windows.bat
```

Si aparece el mensaje «No hay capa curated para SPY», falta correr el pipeline o la variable `ETL_DATOS` apunta a otra carpeta.

Estructura del repositorio

Elemento	Contenido
`config/config.yaml`	Activos, calendarios, fuentes, limpieza, calidad, modelado, backtesting, carga y orquestación.
`R/`	Módulos: utils, config, calendario, limpieza, calidad, transformacion, modelado, backtesting, almacen, carga, extraccion, tareas, sintetico, visualizar.
`pipeline.R`	Punto de entrada del ETL, los modelos y el backtesting.
`ver_*.R` , `visualizar.R`	Consulta de resultados y gráficas.
`tests/`	89 pruebas con testthat.
`programar_windows.bat`	Programación diaria con el Programador de tareas.
`.env.example`	Plantilla de claves y de la variable `ETL_DATOS` .
`README.md`	Guía de uso.